
Metropolis-Hastings Sampling for 3D Gaussian Reconstruction

Hyunjin Kim¹ Haebeom Jung² Jaesik Park²

¹UC San Diego ²Seoul National University
hyunjin@ucsd.edu {haebeom.jung, jaesik.park}@snu.ac.kr

Abstract

We propose an adaptive sampling framework for 3D Gaussian Splatting (3DGS) that leverages comprehensive multi-view photometric error signals within a unified Metropolis-Hastings approach. Traditional 3DGS methods heavily rely on heuristic-based density-control mechanisms (e.g., cloning, splitting, and pruning), which can lead to redundant computations or the premature removal of beneficial Gaussians. Our framework overcomes these limitations by reformulating densification and pruning as a probabilistic sampling process, dynamically inserting and relocating Gaussians based on aggregated multi-view errors and opacity scores. Guided by Bayesian acceptance tests derived from these error-based importance scores, our method substantially reduces reliance on heuristics, offers greater flexibility, and adaptively infers Gaussian distributions without requiring predefined scene complexity. Experiments on benchmark datasets, including Mip-NeRF360, Tanks and Temples, and Deep Blending, show that our approach reduces the number of Gaussians needed, enhancing computational efficiency while matching or modestly surpassing the view-synthesis quality of state-of-the-art models. Our project page is at <https://hjhyunjinkim.github.io/MH-3DGS>.

1 Introduction

Novel view synthesis (NVS) focuses on creating photo-realistic images of a 3D scene from unseen viewpoints and is significant due to its broad applicability across various real-world scenarios. While Neural Radiance Fields (NeRF) [34] has largely advanced Novel View Synthesis by mapping the 3D location and view directions to a view-dependent color and volumetric density, its slow rendering hinders real-world deployment. To address these limitations, 3D Gaussian Splatting (3DGS) [18] recently emerged, enabling real-time photo-realistic rendering by representing complex scenes with explicit 3D Gaussians, each of which is projected to the screen and defined by its position, anisotropic covariance, and opacity.

Despite these advantages, 3DGS and its extensions [18, 55, 16] rely heavily on heuristic-based adaptive density control mechanisms, such as cloning, splitting, and pruning by introducing fixed thresholds on opacity, size, and positional gradients in the view space. This leads to inflation in memory with redundant Gaussians when thresholds are loose or sacrifice fidelity when they are tight. To tackle this problem, Kheradmand et al. [20] recast 3DGS updates as Stochastic Gradient Langevin Dynamics (SGLD), framing Gaussian updates as Markov Chain Monte Carlo (MCMC) samples from an underlying probability distribution representing scene fidelity. As SGLD updates Gaussians in the direction of the image-reconstruction gradient and adds a small noise term, the procedure behaves like a gradient-based optimizer. Although it replaces heuristics with state transitions, it still fixes the total number of Gaussians upfront, limiting adaptability to scenes of varying complexity.

Motivated by these considerations, we propose an adaptive sampling framework that uniquely leverages comprehensive multi-view error signals within a unified Metropolis-Hastings (MH) [33, 7]

approach. We reformulate 3D Gaussian Splatting (3DGS) as an MH sampling process and newly derive a concise, rigorously proven acceptance probability grounded in the classical MH rule. By casting the densification and pruning stages of 3DGS as a probabilistic sampling problem, our method selectively adds Gaussians in regions with high visual or geometric significance as indicated by aggregated view-consistent photometric discrepancies. By deriving Gaussian-level importance scores from opacity and multi-view error metrics, we generate Gaussian candidates through coarse-to-fine proposals, which are subsequently accepted or rejected via Bayesian acceptance tests. This probabilistic, heuristic-free process adaptively converges to a well-balanced Gaussian distribution without requiring prior knowledge of scene complexity, seamlessly integrating with existing 3DGS workflows to achieve superior reconstruction quality, capturing both coarse structures and fine details.

We demonstrate the effectiveness of our approach on real-world scenes from the Mip-NeRF360 [1], Tanks and Temples [21], and Deep Blending [15] datasets. Our method reduces the number of Gaussians needed to represent each scene solely through an improved sampling mechanism, while consistently matching or surpassing the view-synthesis quality of 3DGS across diverse scenes. It also performs on par with, or outperforms Kheradmand et al. [20]. By coupling principled probabilistic inference with dense multi-view error signals, our method matches the rendering accuracy of 3DGS-MCMC [20] with a reduced number of Gaussians, demonstrating that comparable quality can be achieved with a leaner, more generalizable representation.

We summarize our contributions as follows:

- We introduce an adaptive Metropolis–Hastings (MH) framework that replaces densification heuristics in 3DGS with a closed-form Bayesian posterior for each Gaussian and a lightweight photometric surrogate that turns the MH acceptance ratio into a single logistic–voxel product.
- To this end, we derive Metropolis–Hastings transition equations that are specialized to 3DGS and prove that this MH sampler is mathematically sound, demonstrating that the heuristic densification rule in 3DGS can be recast as a principled MH update.
- We show through our experiments that our framework attains state-of-the-art view-synthesis quality with faster convergence.

2 Related Work

2.1 Neural Radiance Fields

Novel View Synthesis generates realistic images from novel viewpoints that differ from the original captures [55]. Neural Radiance Fields (NeRF) [34] have been widely explored in this field because of their remarkable ability to render photorealistic images: an MLP models the 3D scene from multi-view 2D images and enables novel-view generation with multi-view consistency, inspiring extensions that scale to large scenes [13, 50, 44], handle dynamic content [39, 40, 8, 47, 14], generalize to unseen scenes [37, 54], or dispense with known camera poses [31, 17, 25, 32, 2, 56]. Yet the significant limitation of NeRF is that novel-view synthesis demands extensive training and rendering time, prompting many works to accelerate these stages [43, 10, 35, 5, 41, 11].

Building on these advancements, recent research on NeRF explored efficient sampling and optimization to handle complex scenarios. Goli et al. [12] applies the Metropolis-Hastings algorithm to iteratively refine samples iteratively, accurately aligning partially overlapping scenes. Similarly, Bortolon et al. [3] integrates Metropolis-Hastings sampling to select key surface points for efficient ray casting, enabling real-time 6DoF pose estimation without an initial pose. While these methods primarily use Metropolis–Hastings for surface point selection, our work reformulates the entire 3D reconstruction problem as a Metropolis–Hastings sampling process. This perspective allows us to draw Gaussian samples without heuristic assumptions and further underscores the benefits of probabilistic sampling.

2.2 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) [18] recently emerged as a compelling alternative to NeRF by using differentiable rasterization rather than relying on neural networks for volume rendering. 3DGS uses 3D Gaussian primitives to represent scenes, which are initialized from sparse point clouds generated by Structure-from-Motion (SfM) [42]. This representation enables efficient optimization through

interleaved updates of each Gaussian’s position, covariances, opacities, and colors, represented by Spherical Harmonics.

Given 3DGS’s remarkable efficiency and high-resolution output, numerous studies have extended its application across a range of domains, from dynamic scene reconstruction [51, 29, 24, 49, 23], 3D generation [45, 53, 46, 57, 6, 27] to large-scale scene reconstruction [19, 26, 28]. A critical drawback, however, is that each Gaussian adds a significant memory burden due to attributes such as covariance matrices and spherical harmonics (SH) parameters, prompting several memory-efficient variants [22, 9, 36, 48, 38]. Yet these extensions depend on heuristic density-control rules, which constrain efficiency and generality. Addressing this limitation, we introduce a probabilistic framework that replaces fixed thresholds with a unified Metropolis-Hastings sampling strategy, absorbing both densification and pruning in 3DGS and thereby enhancing model efficiency and versatility.

2.3 Adaptive Density Control for 3D Gaussian Splatting

Kheradmand et al. [20] recently introduced the use of Stochastic Gradient Langevin Dynamics (SGLD), a modification of stochastic gradient descent and MCMC, for sampling and optimization in 3DGS, combined with an opacity-driven relocation strategy to shift low-opacity Gaussians towards regions with higher-opacity Gaussians, while enforcing a fixed limit on the total number of points. Additionally, Bulò et al. [4] introduces an error-based densification method that prioritizes areas with high perceptual errors, corrects opacity bias in cloning, and sets a global limit on the number of Gaussians. Similarly, Ye et al. [52] addresses the limitations of gradient collision in adaptive density control by proposing a homodirectional gradient as a criterion for densification. Moreover, Mallick et al. [30] presents a score-based guided densification approach that uses training-time priors to restrict uncontrolled model-size growth. As 3DGS-MCMC [20] is most closely aligned with our objectives and delivers the best overall reconstruction quality, it is our primary baseline for comparison.

In contrast, our proposed framework leverages a unified Metropolis-Hastings sampling strategy grounded in comprehensive multi-view error signals. Rather than relying on thresholds, our method computes Gaussian importance scores from aggregated multi-view photometric errors, dynamically adapting both the number and spatial distribution of Gaussians based on probabilistic inference. We derive the MH acceptance rule in the context of 3DGS and prove that our importance-weighted proposals preserve detailed balance with respect to the multi-view photometric likelihood. This rigorous formulation clearly distinguishes our framework from prior methods addressing this task. Consequently, our approach naturally scales with scene complexity, improving flexibility and efficiency.

3 Preliminary

3.1 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) [18] represents the 3D scene as a set of anisotropic 3D Gaussians, further optimized by differentiable tile rasterization. Each 3D Gaussian $\mathcal{G}$ is characterized by a position vector μ and a covariance matrix Σ :

$$\mathcal{G}(x) = e^{-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)}, \quad \Sigma = RSS^\top R^\top, \quad (1)$$

where the covariance matrix is factorized into a scaling matrix S and a rotation matrix R . To optimize the parameters of the 3D Gaussians for an accurate scene representation, it is essential to render them into images using a differentiable process. The transformed covariance matrix Σ' , now in 2D, is calculated by the viewing transform W and the Jacobian of the affine transformation of the projective transformation J as $\Sigma' = JW\Sigma W^\top J^\top$. The color C of a pixel can be computed by blending $\mathcal{N}$ ordered points overlapping the pixel as $C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$, where c_i and α_i represent the view-dependent color and the opacity of the point. These values are computed from a 2D Gaussian distribution with a covariance matrix Σ , scaled by a per-point optimized 3D Gaussian opacity.

Furthermore, to achieve a high-quality representation, 3DGS employs adaptive density control, a heuristic-rule-based strategy that fills under-reconstructed areas by cloning Gaussians and enhances over-reconstructed regions by dividing larger Gaussians. During each refinement step, 3DGS evaluates Gaussians based on their position, opacity, and size, removing those with low opacity or excessive size. For densification, 3DGS assesses the positional gradient $\nabla_p \mathcal{L}$ of each Gaussian, refining further if the

gradient exceeds a predefined threshold τ_p . If a Gaussian surpasses the size threshold τ_s , indicating over-reconstruction, it is split into smaller Gaussians. Alternatively, under-reconstructed areas prompt Gaussian cloning by shifting along the gradient direction, effectively expanding Gaussian coverage for better scene representation.

4 Method

Our method begins with a key design choice: unlike 3DGS, which densifies points once a fixed threshold is reached, we employ a lean point-based sampler that keeps the most informative Gaussians, sharply reducing storage while preserving high-fidelity reconstructions. Sec.4.1 lays the theoretical groundwork for this: we extend the classical Metropolis–Hastings algorithm to 3DGS and rigorously derive scene-adaptive equations that generalize the traditional formulation. Sec.4.2 describes how the sampler proposes candidates guided by multi-view photometric and opacity errors, and Sec.4.3 presents the MH acceptance test, where we newly derive and prove a concise, practical expression for the MH acceptance probability. The complete algorithmic flow is summarized in Alg.1 and 2 in Appendix F.

4.1 Formulation of the Metropolis-Hastings Framework

Our main objective is eliminating heuristic dependence in adaptive density control by recasting densification and pruning as a principled sampling problem. Specifically, we interpret each modification of the Gaussian set as a proposal in a Metropolis-Hastings Markov Chain. (See Appendix E for the preliminary on the Metropolis-Hastings algorithm) Each step in the chain perturbs the current scene representation Θ by (i) **inserting** new Gaussians near pixels that still exhibit high multi-view error or (ii) **relocating** Gaussians whose opacity has collapsed. The proposed scene Θ' is then accepted or rejected according to a probabilistic rule that, in expectation, favors configurations that reproduce the captured images better while remaining spatially sparse.

In our formulation, a scene is described as a collection $\Theta = \{g_i\}_{i=1}^N$ of Gaussian splats, where each splat contains parameters $(\mathbf{x}_i, \Sigma_i, c_i, \alpha_i)$, representing the position, covariance, color, and opacity of a Gaussian. We define the overall loss function as:

$$\mathcal{L}(\Theta) = (1 - \lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{D-SSIM}} + \lambda_{\text{opacity}}\bar{\alpha} + \lambda_{\text{scale}}\bar{\Sigma} \quad (2)$$

where $\mathcal{L}_1$ and $\mathcal{L}_{\text{D-SSIM}}$ are the loss terms from the original 3DGS [18] method. The last two terms are regularizers on the mean opacity and covariance, used to penalize large or widespread opacities and large Gaussian spreads, respectively, as advocated by 3DGS-MCMC [20]. We use $\lambda = 0.2$, $\lambda_{\text{opacity}} = 0.01$ and $\lambda_{\text{scale}} = 0.01$ in our framework.

From a likelihood viewpoint, we treat the minimization of $\mathcal{L}(\Theta)$ equivalent to maximizing the likelihood of the observed image set $\mathcal{D} = \{I^{(v)}\}$. Thus, we can interpret this as:

$$\ln p(\mathcal{D} | \Theta) \approx -\mathcal{L}(\Theta) \quad (3)$$

Since likelihood terms are usually expressed with base e , we adopt the natural logarithm ($\ln$) throughout to keep the formulas clean and consistent.

Furthermore, photometric fidelity alone does not stop the optimizer from piling Gaussians onto the same patch once it converges locally. Thus, we discourage such redundancies by introducing a voxel-wise prior:

$$\ln p(\Theta) = -\lambda_v \sum_{v \in \mathcal{V}} \ln(1 + c_{\Theta}(v)) \quad (4)$$

where $c_{\Theta}(v)$ is the number of Gaussians in voxel v . This prior encourages at most one or two Gaussians per voxel. Since the cost grows logarithmically, empty cells get almost no penalty, but the penalty rises quickly once the voxel becomes crowded. Combining Eq. 2 and 4, we are able to define a negative log-posterior:

$$\mathcal{E}(\Theta) = \mathcal{L}(\Theta) + \lambda_v \sum_{v \in \mathcal{V}} \ln(1 + c_{\Theta}(v)) \quad (5)$$

Based on this derivation, we can define the posterior density based on Bayesian statistics:

$$\pi(\Theta) = \frac{1}{Z} e^{-\mathcal{E}(\Theta)}, \quad (6)$$

where Z is a normalization constant (See Appendix A for details). Our framework samples Gaussian configurations Θ in proportion to $\pi(\Theta)$, adaptively refining the 3D Gaussian representation to cover undersampled regions. To facilitate this sampling, we construct a Markov Chain whose states are successive scene representations $\Theta_0, \Theta_1, \dots, \Theta_t$. From a given state Θ_t , we draw a proposal Θ' by adding Gaussians in high-error areas or relocating low opacity Gaussians, with probability density $q(\Theta' | \Theta_t)$. The construction of $q(\Theta' | \Theta_t)$ is detailed in Section 4.2.

We accept or reject Θ' according to the Metropolis-Hastings (MH) rule:

$$\rho(\Theta' | \Theta_t) = \min\left(1, \frac{\pi(\Theta') q(\Theta_t | \Theta')}{\pi(\Theta_t) q(\Theta' | \Theta_t)}\right), \quad \rho(\Theta' | \Theta_t) \in [0, 1] \quad (7)$$

where $\rho(\Theta' | \Theta_t)$ indicates the standard MH acceptance probability. At each step we draw $u \sim \mathcal{U}(0, 1)$; if $u < \rho$, the proposal is accepted and we set $\Theta_{t+1} = \Theta'$, otherwise we keep $\Theta_{t+1} = \Theta_t$. We explain how the MH rule accepts or rejects Θ' again in Section 4.3. Over many iterations, this Markov Chain converges toward our target distribution $\pi(\Theta)$, systematically densifying the Gaussian set where it reduces loss and relocating Gaussians that fail to improve image fidelity.

4.2 Proposal Generation

Building on the Metropolis-Hastings framework, we now specify how a candidate configuration Θ' is generated from the current state Θ . The guiding principle is to concentrate proposals in problematic regions where Θ either (1) lacks sufficient **opacity**, indicating missing or overly sparse geometry, or (2) exhibits a large **photometric error**.

4.2.1 Per-pixel importance field

At every densification step, we first choose a working camera subset $\mathcal{C}_t = \{c_{t,1}, \dots, c_{t,k_t}\}$ from the full training set $\mathcal{C}$. The subset size is annealed according to

$$k_t = \max\left(1, \lfloor (1 - \eta_t) |\mathcal{C}| \rfloor\right), \quad \eta_t = \frac{t - t_{\min}}{t_{\max} - t_{\min}} \quad (8)$$

so that early iterations ($\eta_t \approx 0$) ensure broad coverage, whereas we reduce the subset size to focus on specific vantage points requiring finer coverage. We use a *round-robin* schedule that walks through $\mathcal{C}$ in contiguous blocks of length k_t , as this allows every viewpoint to contribute equally to the error signal after a few iterations. For each camera $c \in \mathcal{C}_t$, we compare the current prediction $\hat{I}^{(c)}$ to the ground truth image $I^{(c)}$ and record per-pixel SSIM and $\mathcal{L}_1$ errors. Averaging over the subset yields multi-view aggregate maps of:

$$\text{SSIM}_{\text{agg}}(p) = \frac{1}{k_t} \sum_{c \in \mathcal{C}_t} [1 - \text{SSIM}(I^{(c)}(p), \hat{I}^{(c)}(p))], \quad \mathcal{L}_{1\text{agg}}(p) = \frac{1}{k_t} \sum_{c \in \mathcal{C}_t} |I^{(c)}(p) - \hat{I}^{(c)}(p)| \quad (9)$$

where k is the number of selected cameras, and p indicated image pixels. This setup ensures we capture *multi-view* errors rather than relying on a single viewpoint. We then feed $\text{SSIM}_{\text{agg}}(p)$ and $\mathcal{L}_{1\text{agg}}(p)$ along with the per-pixel opacity $O(p)$ into our importance score. After applying robust normalization to each quantity (to mitigate outliers and match dynamic ranges), the three cues are fused through a logistic function:

$$s(p) = \sigma\left(\alpha O(p) + \beta \text{SSIM}_{\text{agg}}(p) + \gamma \mathcal{L}_{1\text{agg}}(p)\right), \quad \sigma(z) = \frac{1}{1 + e^{-z}} \quad (10)$$

where α, β, γ control the relative emphasis on opacity, structural similarity, and photometric fidelity, respectively. We use $\alpha = 0.8, \beta = 0.5, \gamma = 0.5$ in our tests. The scalar field $s \in (0, 1)$ highlights the pixels that are simultaneously under-covered and photometrically inaccurate across the chosen views.

4.2.2 Mapping Pixels to Gaussian Importance

Given the 3D position $\mathbf{x}_i \in \mathbb{R}^3$ of a Gaussian g_i , we obtain its image-plane coordinates via the calibrated projection $\Pi: \mathbb{R}^3 \rightarrow \mathbb{R}^2$. Rather than integrating the importance field over the full elliptical footprint of g_i , which requires evaluating hundreds of pixels per splat, we adopt a lightweight surrogate that uses the floor operator to sample only the pixel lying directly beneath the center of g_i :

$$I(i) = s(\lfloor \Pi(\mathbf{x}_i) \rfloor) \quad (11)$$

where $s(\cdot)$ is the logistic importance map in Eq. 10. The importance weight $I(i)$ is then normalized once per iteration to give a categorical distribution $P_{\text{pick}}(i) = I(i) / \sum_j I(j)$.

4.2.3 Proposal Distribution

Having established a Bayesian viewpoint on our 3D Gaussian representation and a per-pixel importance score $s(p)$, we now detail the proposal mechanism in our Metropolis-Hastings (MH) sampler. We maintain a Markov Chain $\Theta_0 \rightarrow \Theta_1 \rightarrow \dots$, each Θ_t comprising the current Gaussian set. From state Θ_t , we form a proposal Θ' in **two** phases.

During the **Coarse Phase**, we sample a batch $\mathcal{I}_c = \{i_1^c, \dots, i_{B_c}^c\}$ of size B_c from P_{pick} . For every index $i \in \mathcal{I}_c$, we create a new Gaussian:

$$g_i^c = (\mathbf{x}_i + \delta_i^c, \Sigma_i, \mathbf{c}_i, \alpha_i), \quad \delta_i^c \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{coarse}}^2 \mathbf{I}) \quad (12)$$

where σ_{coarse} is chosen to be large enough to enable the sampler to explore broad coverage gaps.

In the **Fine Phase**, a second batch $\mathcal{I}_f = \{i_1^f, \dots, i_{B_f}^f\}$ is also drawn from the distribution P_{pick} . Similar to the coarse phase, we create a new Gaussian for every index in $i \in \mathcal{I}_f$:

$$g_i^f = (\mathbf{x}_i + \delta_i^f, \Sigma_i, \mathbf{c}_i, \alpha_i), \quad \delta_i^f \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{fine}}^2 \mathbf{I}) \quad (13)$$

where $\sigma_{\text{fine}} < \sigma_{\text{coarse}}$. The smaller perturbation radius focuses on residual high-importance regions that still yield large weights $I(i)$ (and hence a large $P_{\text{pick}}(i)$) after the coarse phase fills the major coverage gaps. This allows our method to refine the geometry further with locally placed Gaussians.

Finally, all the newly spawned Gaussians are combined to form the birth component Θ'_{birth} of the proposal. This is combined the current state Θ_t to yield a full proposal Θ' :

$$\Theta'_{\text{birth}} = \{g_i^c \mid i \in \mathcal{I}_c\} \cup \{g_i^f \mid i \in \mathcal{I}_f\}, \quad \Theta' = \Theta_t \cup \Theta'_{\text{birth}} \quad (14)$$

Formally, the probability of drawing the birth component is:

$$q(\Theta' \mid \Theta) = \prod_{i \in \mathcal{I}_c \cup \mathcal{I}_f} P_{\text{pick}}(i) \mathcal{N}(\mathbf{x}'_i; \mathbf{x}_i, \sigma_{\phi}^2 \mathbf{I}) \quad (15)$$

4.2.4 Relocation of Low Contributing Gaussians

In addition to densification, our framework incorporates a relocation mechanism inspired by 3DGS-MCMC [20] to reuse Gaussians with persistently low opacity. Let $\mathcal{D} = \{i \mid \alpha_i \leq \tau\}$ denote the set of Gaussians with low opacity, and $\mathcal{A} = \{j \mid \alpha_j > \tau\}$ the set of high opacity Gaussians. We consider $\tau = 0.005$. For each $g_i \in \mathcal{D}$, we sample a replacement index $j \in \mathcal{A}$ with probability

$$p(j) = \frac{\alpha_j}{\sum_{k \in \mathcal{A}} \alpha_k},$$

and update g_i 's parameters by transferring those from g_j , optionally modulated by a ratio reflecting the sampling frequency. The relocation step preserves the representational fidelity of our framework by continuously reinforcing effective Gaussians while reassigning under-contributing ones.

4.3 Metropolis-Hastings Proposal Acceptance

Now, the Metropolis-Hastings sampler must decide whether the proposal Θ' is kept or rejected. Recall the MH rule in Eq. 7. Considering the posterior density in Eq. 6, we can rewrite the rule as:

$$\rho_{\text{MH}} = \min \left\{ 1, e^{-\Delta \mathcal{E}} \frac{q(\Theta \mid \Theta')}{q(\Theta' \mid \Theta)} \right\} \quad (16)$$

where $\Delta \mathcal{E} = \mathcal{E}(\Theta') - \mathcal{E}(\Theta)$. The proposal Θ' is a batch of new Gaussians, but since the perturbations are independent, we can evaluate the acceptance separately. Considering one candidate Gaussian whose center falls in voxel v' , the insertion of that particular Gaussian leads to $\Delta \mathcal{E}$ as:

$$\Delta \mathcal{E} = \Delta \mathcal{L} + \lambda_v \ln \left(\frac{1 + c_{\Theta}(v') + 1}{1 + c_{\Theta}(v')} \right) = \Delta \mathcal{L} - \ln D(v'), \quad D(v') = \frac{1}{1 + \lambda_v c_{\Theta}(v')} \quad (17)$$

where $\Delta \mathcal{L}$ is the photometric change and $c(v')$ is the voxel occupancy before the insertion. (See Appendix B for details.) However, evaluating $\mathcal{L}$ requires rendering every view in the camera set $\mathcal{C}_t$, which is computationally expensive. Thus, we use the tight empirical correlation between the importance map $I(i)$ and the loss reduction, $-\Delta \mathcal{L} \approx I(i)$, as a surrogate. (See Appendix C for details.)

Moreover, the calculation of $q(\Theta \mid \Theta')$ in Eq. 16, requires the enumeration of the reverse action, which is deleting the particular newborn Gaussian. As our framework does not generate such move, we absorb the unavailable reverse-proposal density into the voxel factor $D(v') = (1 + \lambda_v c_{\Theta}(v'))^{-1}$, which already penalises insertions in crowded cells. Substituting both surrogates into Eq. 16 gives an upper bound $\min\{1, e^{I(i)} D(v')\}$, and mapping the exponential through the logistic $\sigma(z) = (1 + e^{-z})^{-1}$ yields a practical version of the MH rule:

$$\rho(i) = \sigma(I(i)) D(v') \quad (18)$$

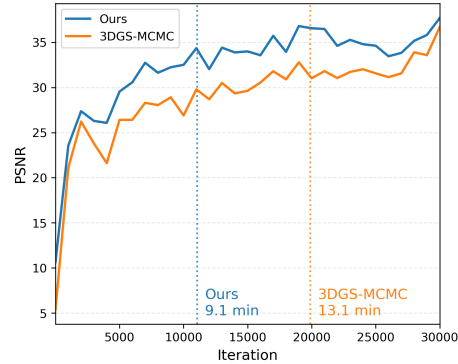
Following the Metropolis-Hastings algorithm, we draw a uniform random number $u \sim \mathcal{U}(0, 1)$ for each candidate Gaussian. The proposal is accepted and added to the current set Θ_t when $u < \rho(i)$, otherwise it is rejected. Because $\rho(i) = \sigma(I(i)) D(v')$ is large only when the per-Gaussian importance $I(i)$ is high and the voxel factor $D(v')$ is close to one, Gaussians that target significant photometric errors in sparsely populated regions are accepted with high probability, whereas candidates in already crowded voxels or low-error areas are mostly discarded.

4.4 Comparison with 3DGS-MCMC [20]

Although both methods fall under the broad scope of Monte-Carlo sampling, they embody *fundamentally different* MCMC philosophies. 3DGS-MCMC treats every Gaussian parameter as a latent variable and applies SGLD: it perturbs all dimensions with small, isotropic noise and accepts every proposed move, so the chain drifts locally through the existing cloud.

In contrast, our method runs a Metropolis-Hastings birth sampler that targets only positions. We draw global proposals straight in image-space voids highlighted by a multi-view error map, then pass each candidate through a Metropolis-Hastings accept-reject test that weighs its predicted photometric gain against a sparsity prior. This decouples exploration (“where to propose”) from retention (“what to keep”), enabling long jumps to uncovered regions and the principled dismissal of redundant splats, which Langevin-noise walks with unconditional acceptance cannot achieve. Moreover, we analytically derive and rigorously prove that the 3DGS densification process can be recast as a Metropolis-Hastings sampler, establishing a principled bridge between 3DGS and our formulation. In principle, 3DGS-MCMC is a local, always-accept SGLD chain, whereas our approach is a global, importance-driven MH chain focused on the scene’s unexplained areas. This leads to our method achieving faster convergence, see Fig. 1 for the comparison.

Figure 1: PSNR over training iterations for our method and 3DGS-MCMC [20] on the Deep Blending [15] dataset. Dotted vertical lines mark the first iteration and the wall-clock time at which each method reaches 95 % of its eventual PSNR. Our method converges appreciably faster, and also attains a higher final PSNR.



5 Experiments

5.1 Experimental Settings

Dataset and Metrics. We perform comprehensive experiments using the same real-world datasets employed in 3DGS. Specifically, we utilize the scene-scale view synthesis dataset from Mip-NeRF360 [1], which includes nine large-scale real-world scenes comprising five outdoor and four indoor environments. Additionally, we chose two scenes from the Tanks and Temples [21] dataset and two from the Deep Blending [15] dataset, focusing on the same scenes used in the original 3DGS. We evaluate each method using the standard evaluation metrics: peak signal-to-noise ratio (PSNR), structural similarity index measure (SSIM), and learned perceptual image patch similarity (LPIPS).

Implementation Details. Our framework is based on the original 3DGS [18] implementation. We also use the differentiable tile rasterizer implementation provided by 3DGS-MCMC [20]. We conducted experiments on an NVIDIA GeForce RTX 3090 GPU. Averaged across all datasets, our method runs at 0.0528 s per iteration, and 26.4 min for 30,000 iterations. During densification, a

Table 1: Quantitative results of our method and state-of-the-art models evaluated on Mip-NeRF 360 [1], Tanks&Temples [21] and Deep Blending [15] datasets. We present the baseline results from 3DGS [18]. For a fair comparison, we also report the results for 3DGS obtained from our experiments (Denoted as 3DGS*) and the results from 3DGS and 3DGS-MCMC trained with the *same number of Gaussians* as our method (denoted as 3DGS-S and 3DGS-MCMC, respectively). The unit of the number of Gaussians is million. The results are colored as **best**, **second-best**, and **third-best**. To avoid redundancy, only 3DGS* results are considered instead of 3DGS.

Dataset	Mip-NeRF 360				Tanks & Temples				Deep Blending			
Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	# GS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	# GS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	# GS $\downarrow$
Plenoxels [10]	23.08	0.626	0.463	-	21.08	0.719	0.379	-	23.06	0.795	0.510	-
INGP-base [35]	25.30	0.671	0.371	-	21.72	0.723	0.330	-	23.62	0.797	0.423	-
INGP-big [35]	25.59	0.699	0.331	-	21.92	0.745	0.305	-	24.96	0.817	0.390	-
Mip-NeRF 360 [1]	27.69	0.792	0.237	-	22.22	0.759	0.257	-	29.40	0.901	0.245	-
3DGS [18]	27.21	0.815	0.214	-	23.14	0.841	0.183	-	29.41	0.903	0.243	-
3DGS* [18]	27.44	0.811	0.223	3.176	23.63	0.848	0.177	1.831	29.53	0.904	0.244	2.815
3DGS-S [18]	26.62	0.761	0.299	0.723	23.38	0.828	0.214	0.773	29.27	0.897	0.272	0.751
3DGS-MCMC [20]	27.69	0.816	0.234	0.723	24.30	0.862	0.170	0.773	29.84	0.906	0.254	0.751
Ours	27.34	0.798	0.241	0.723	23.99	0.852	0.166	0.773	30.12	0.909	0.245	0.751

Table 2: Ablations on the proposed contributions for our Per-Gaussian Importance Score calculation. We test on the Tanks & Temples [21] - Truck and Deep Blending [15] - Dr Johnson dataset.

Method \ Dataset			Truck		Dr Johnson	
SSIM	L1	Opacity	PSNR $\uparrow$	# of Gaussians $\downarrow$	PSNR $\uparrow$	# of Gaussians $\downarrow$
			25.37	2,575,516	29.08	3,308,886
✓	✓		25.47	756,180	29.47	766,421
✓	✓	✓	25.54	756,104	29.64	766,469

normalized progress value ($\in [0, 1]$) linearly transitions parameters from coarse (larger offsets, voxel sizes, and batch sizes) to fine (smaller offsets and voxel sizes) to shift focus from broad to precise refinements. Coarse proposal scales range from 10.0 to 5.0, fine proposal scales from 2.0 to 1.0, and voxel sizes from 0.02 to 0.005. Batch sizes for coarse and fine proposals are set at 4,500 and 16,000.

Comparison with Baselines. We perform comparisons with methodologies that model large-scale scenes. We compare our approach against state-of-the-art baselines, such as Plenoxels [10], Instant-NGP [35], Mip-NeRF 360 [1], and 3DGS [18]. For the baselines aside from 3DGS [18], we present the original results as reported in 3DGS [18]. Moreover, we include a comparison with 3DGS-MCMC [20], which has a similar motivation and shows state-of-the-art performance.

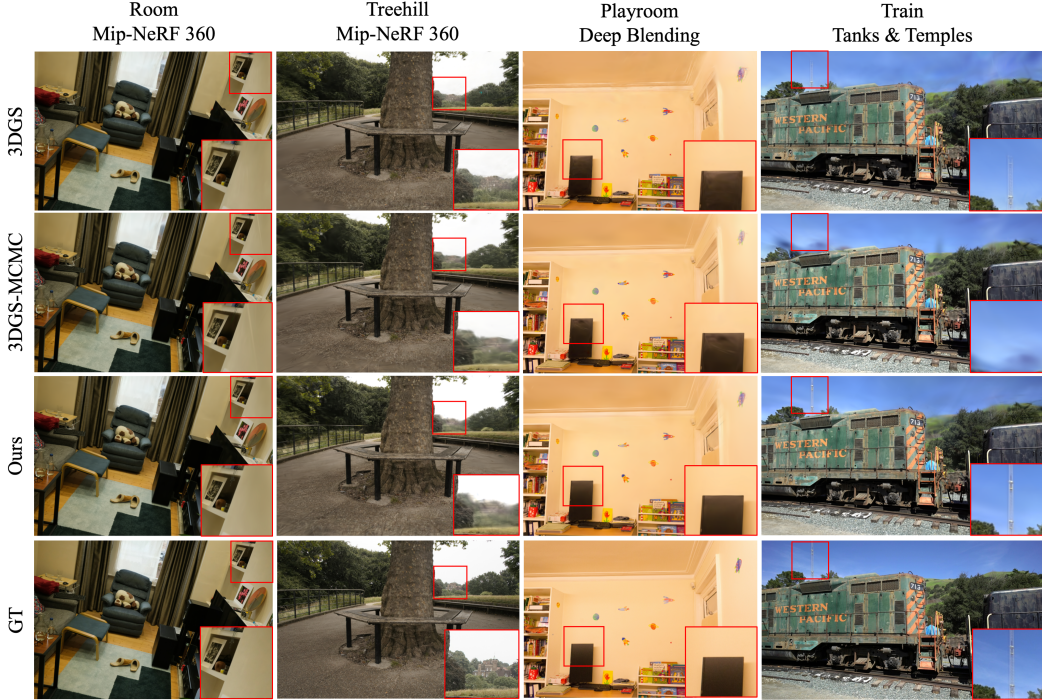
5.2 Results

Table 1 presents the quantitative evaluations, and Fig 2 demonstrates the qualitative evaluations on real-world scenes. Our method consistently surpasses the performance of 3DGS when using the same number of Gaussians, and achieves or exceeds the performance of the original 3DGS while using fewer points than 3DGS. Unlike 3DGS, which relies on fixed thresholds for densification, our framework leverages a streamlined point-based sampling strategy that drastically reduces storage requirements while preserving high-fidelity reconstructions. In particular, Table 4 in Appendix D shows how our approach maintains a consistent, efficient sampling rate across scenes of varying complexity, underscoring its versatility. We also provide the full per-scene evaluations in Appendix G. Although 3DGS-MCMC can sometimes yield slightly higher metrics, it updates and optimizes every Gaussian parameter, which increases its inefficiency and necessitates knowing the scene’s complexity (the number of Gaussians) in advance. In contrast, our method focuses exclusively on sampling and optimizing point positions, providing a simpler yet equally accurate alternative. As evidenced by Fig. 1, our method reaches the 95 % PSNR threshold faster than 3DGS-MCMC in the Deep Blending [15] dataset, both in iteration and wall-clock time, demonstrating faster convergence. This shows that our framework can still achieve state-of-the-art reconstruction quality even without exhaustive parameter updates.

5.3 Ablation Study

Components for the Importance Scores. We conducted an ablation study on the key Gaussian components to identify the most influential factors in importance score calculation for accurate sam-

Figure 2: Qualitative results of our method compared to 3DGS and the corresponding ground truth image from held-out test views. As seen in the red bounding box, our method can better capture the fine and coarse details that 3DGS [18] and 3DGS-MCMC [20] miss.



pling. The results, shown in Table 2, support the effectiveness of our *Importance Score Computation* design. Specifically, combining SSIM, L1 loss, and opacity values leads to higher PSNR scores and reduces the required number of Gaussians compared to both the baseline and variants that rely only on photometric losses. This study underscores the need for careful factor selection.

Effectiveness of Relocation. We explore whether relocating Gaussians was effective in Table 3. Our experiments confirm that the relocation strategy is essential for maintaining high representational fidelity. It shows that dynamically reallocating underperforming Gaussians to areas where they contribute more effectively improves performance in both indoor and outdoor scenes.

Table 3: Ablation on relocation. Results are presented for both 3DGS and 3DGS-S, the latter using the same number of Gaussians as ours.

	Mip-NeRF360 [1] - Bonsai	T&T [21] - Train
	PSNR↑ / SSIM↑ / LPIPS↓	PSNR↑ / SSIM↑ / LPIPS↓
3DGS [18]	32.14 / 0.940 / 0.206	21.89 / 0.814 / 0.207
3DGS-S [18]	31.37 / 0.935 / 0.205	21.80 / 0.791 / 0.245
Ours w/o Reloc.	32.19 / 0.946 / 0.181	22.16 / 0.811 / 0.219
Ours	32.52 / 0.950 / 0.171	22.44 / 0.821 / 0.198

6 Conclusion

In this paper, we present an adaptive sampling framework that unifies densification and pruning for 3D Gaussian Splatting via a probabilistic Metropolis-Hastings scheme. The importance-weighted sampler automatically targets visually or geometrically critical regions while avoiding redundancy. We derive a new and concise probabilistic formulation that links proposals to scene importance, ensuring faster convergence without sacrificing accuracy. Our experiments show that our method matches or surpasses state-of-the-art quality with fewer Gaussians, and our method’s simplicity invites further advances in 3D scene representation.

Limitations and Future Work. Our method is not without limitations. Our Metropolis-Hastings sampler, though aware of low-density regions, still prioritizes high-density areas, leading to outdoor scenes with highly sparse data performing less well than indoor scenes. Moreover, as we use multi-view photometric errors when proposing points, it adds roughly 5 minutes of extra training time per scene. Crafting proposal distributions tuned for sparse regions and introducing faster optimization to cut the training time would be an exciting direction for future research.

References

- [1] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields. *CVPR*, 2022.
- [2] Wenjing Bian, Zirui Wang, Kejie Li, Jiawang Bian, and Victor Adrian Prisacariu. NoPe-NeRF: Optimising Neural Radiance Field with No Pose Prior. 2023.
- [3] Matteo Bortolon, Theodore Tsismelis, Stuart James, Fabio Poiesi, and Alessio Del Bue. IFFNeRF: Initialization Free and Fast 6DoF pose estimation from a single image and a NeRF model. 2024.
- [4] Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in gaussian splatting. *arXiv preprint arXiv:2404.06109*, 2024.
- [5] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. TensorRF: Tensorial Radiance Fields. In *European Conference on Computer Vision (ECCV)*, 2022.
- [6] Zilong Chen, Feng Wang, Yikai Wang, and Huaping Liu. Text-to-3D using Gaussian Splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21401–21412, June 2024.
- [7] Siddhartha Chib and Edward Greenberg. Understanding the Metropolis-Hastings Algorithm. *The American Statistician*, 49(4):327–335, 1995. doi: 10.1080/00031305.1995.10476177.
- [8] Yilun Du, Yinan Zhang, Hong-Xing Yu, Joshua B. Tenenbaum, and Jiajun Wu. Neural Radiance Flow for 4D View Synthesis and Video Processing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021.
- [9] Zhiwen Fan, Kevin Wang, Kairun Wen, Zehao Zhu, Dejia Xu, and Zhangyang Wang. LightGaussian: Unbounded 3D Gaussian Compression with 15x Reduction and 200+ FPS, 2023.
- [10] Sara Fridovich-Keil, Alex Yu, Matthew Tancik, Qinhong Chen, Benjamin Recht, and Angjoo Kanazawa. Plenoxels: Radiance Fields Without Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5501–5510, June 2022.
- [11] Stephan J Garbin, Marek Kowalski, Matthew Johnson, Jamie Shotton, and Julien Valentin. Fastnerf: High-fidelity neural rendering at 200fps. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 14346–14355, 2021.
- [12] Lily Goli, Daniel Rebain, Sara Sabour, Animesh Garg, and Andrea Tagliasacchi. nerf2nerf: Pairwise registration of neural radiance fields. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 2023.
- [13] Jonathan Granskog, Till N. Schnabel, Fabrice Rousselle, and Jan Novák. Neural Scene Graph Rendering. *ACM Transactions on Graphics (Proceedings of SIGGRAPH)*, 40(4), August 2021.
- [14] Yudong Guo, Keyu Chen, Sen Liang, Yong-Jin Liu, Hujun Bao, and Juyong Zhang. Ad-nerf: Audio driven neural radiance fields for talking head synthesis. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5784–5794, 2021.
- [15] Peter Hedman, Julien Philip, True Price, Jan-Michael Frahm, George Drettakis, and Gabriel Brostow. Deep Blending for Free-viewpoint Image-based Rendering. 37(6):257:1–257:15, 2018.
- [16] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D Gaussian Splatting for Geometrically Accurate Radiance Fields. In *SIGGRAPH 2024 Conference Papers*. Association for Computing Machinery, 2024. doi: 10.1145/3641519.3657428.
- [17] Yoonwoo Jeong, Seokjun Ahn, Christopher Choy, Animashree Anandkumar, Minsu Cho, and Jaesik Park. Self-calibrating neural radiance fields. In *Proceedings of the Int. Conf. on Computer Vision (ICCV)*, 2021.
- [18] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 42(4), July 2023. URL <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>.
- [19] Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. A Hierarchical 3D Gaussian Representation for Real-Time Rendering of Very Large Datasets. *ACM Transactions on Graphics*, 43(4), July 2024. URL <https://repo-sam.inria.fr/fungraph/hierarchical-3d-gaussians/>.

- [20] Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. Spotlight Presentation.
- [21] Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. Tanks and Temples: Benchmarking Large-Scale Scene Reconstruction. *ACM Transactions on Graphics*, 36(4), 2017.
- [22] Joo Chan Lee, Daniel Rho, Xiangyu Sun, Jong Hwan Ko, and Eunbyung Park. Compact 3D Gaussian Representation for Radiance Field. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21719–21728, 2024.
- [23] Junoh Lee, ChangYeon Won, Hyunjun Jung, Inhwon Bae, and Hae-Gon Jeon. Fully Explicit Dynamic Gaussian Splatting. In *Proceedings of the Neural Information Processing Systems*, 2024.
- [24] Zhan Li, Zhang Chen, Zhong Li, and Yi Xu. Spacetime Gaussian Feature Splatting for Real-Time Dynamic View Synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8508–8520, June 2024.
- [25] Chen-Hsuan Lin, Wei-Chiu Ma, Antonio Torralba, and Simon Lucey. BARF: Bundle-Adjusting Neural Radiance Fields. In *IEEE International Conference on Computer Vision (ICCV)*, 2021.
- [26] Jiaqi Lin, Zhihao Li, Xiao Tang, Jianzhuang Liu, Shiyong Liu, Jiayue Liu, Yangdi Lu, Xiaofei Wu, Songcen Xu, Youliang Yan, and Wenming Yang. VastGaussian: Vast 3D Gaussians for Large Scene Reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5166–5175, June 2024.
- [27] Huan Ling, Seung Wook Kim, Antonio Torralba, Sanja Fidler, and Karsten Kreis. Align Your Gaussians: Text-to-4D with Dynamic 3D Gaussians and Composed Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8576–8588, June 2024.
- [28] Yang Liu, Chuanchen Luo, Lue Fan, Naiyan Wang, Junran Peng, and Zhaoxiang Zhang. Citygaussian: Real-time high-quality large-scale scene rendering with gaussians. In *European Conference on Computer Vision*, pages 265–282. Springer, 2025.
- [29] Zhicheng Lu, Xiang Guo, Le Hui, Tianrui Chen, Ming Yang, Xiao Tang, Feng Zhu, and Yuchao Dai. 3D Geometry-aware Deformable Gaussian Splatting for Dynamic View Synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [30] Saswat Subhajiyo Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia 2024 Conference Papers*, pages 1–11, 2024.
- [31] Ricardo Martin-Brualla, Noha Radwan, Mehdi S. M. Sajjadi, Jonathan T. Barron, Alexey Dosovitskiy, and Daniel Duckworth. NeRF in the Wild: Neural Radiance Fields for Unconstrained Photo Collections. In *CVPR*, 2021.
- [32] Quan Meng, Anpei Chen, Haimin Luo, Minye Wu, Hao Su, Lan Xu, Xuming He, and Jingyi Yu. GNeRF: GAN-based Neural Radiance Field without Posed Camera. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [33] Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The journal of chemical physics*, 21(6): 1087–1092, 1953.
- [34] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *ECCV*, 2020.
- [35] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM transactions on graphics (TOG)*, 41(4):1–15, 2022.
- [36] Simon Niedermayr, Josef Stumpfegger, and Rüdiger Westermann. Compressed 3D Gaussian Splatting for Accelerated Novel View Synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10349–10358, June 2024.
- [37] Michael Niemeyer and Andreas Geiger. Giraffe: Representing scenes as compositional generative neural feature fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11453–11464, 2021.

- [38] Panagiotis Papantonakis, Georgios Kopanas, Bernhard Kerbl, Alexandre Lanvin, and George Drettakis. Reducing the Memory Footprint of 3D Gaussian Splatting. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 7(1), May 2024. URL https://repo-sam.inria.fr/fungraph/reduced_3dgs/.
- [39] Keunhong Park, Utkarsh Sinha, Jonathan T. Barron, Sofien Bouaziz, Dan B Goldman, Steven M. Seitz, and Ricardo Martin-Brualla. Nerfies: Deformable Neural Radiance Fields. *ICCV*, 2021.
- [40] Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T. Barron, Sofien Bouaziz, Dan B Goldman, Ricardo Martin-Brualla, and Steven M. Seitz. HyperNeRF: A Higher-Dimensional Representation for Topologically Varying Neural Radiance Fields. *ACM Trans. Graph.*, 40(6), dec 2021.
- [41] Christian Reiser, Songyou Peng, Yiyi Liao, and Andreas Geiger. KiloNeRF: Speeding up Neural Radiance Fields with Thousands of Tiny MLPs. In *International Conference on Computer Vision (ICCV)*, 2021.
- [42] Noah Snavely, Steven M Seitz, and Richard Szeliski. Photo tourism: exploring photo collections in 3D. In *ACM siggraph 2006 papers*, pages 835–846. 2006.
- [43] Cheng Sun, Min Sun, and Hwann-Tzong Chen. Direct Voxel Grid Optimization: Super-fast Convergence for Radiance Fields Reconstruction. In *CVPR*, 2022.
- [44] Matthew Tancik, Vincent Casser, Xincheng Yan, Sabeek Pradhan, Ben Mildenhall, Pratul P. Srinivasan, Jonathan T. Barron, and Henrik Kretzschmar. Block-NeRF: Scalable Large Scene Neural View Synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8248–8258, June 2022.
- [45] Jiaxiang Tang, Jiawei Ren, Hang Zhou, Ziwei Liu, and Gang Zeng. DreamGaussian: Generative Gaussian Splatting for Efficient 3D Content Creation. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=UyNXMqnN3c>.
- [46] Jiaxiang Tang, Zhaoxi Chen, Xiaokang Chen, Tengfei Wang, Gang Zeng, and Ziwei Liu. LGM: Large Multi-view Gaussian Model for High-Resolution 3D Content Creation. In Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol, editors, *Computer Vision – ECCV 2024*, pages 1–18, Cham, 2025. Springer Nature Switzerland. ISBN 978-3-031-73235-5.
- [47] Edgar Tretschk, Ayush Tewari, Vladislav Golyanik, Michael Zollhöfer, Christoph Lassner, and Christian Theobalt. Non-Rigid Neural Radiance Fields: Reconstruction and Novel View Synthesis of a Dynamic Scene From Monocular Video. In *IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2021.
- [48] Henan Wang, Hanxin Zhu, Tianyu He, Runsen Feng, Jiajun Deng, Jiang Bian, and Zhibo Chen. End-to-End Rate-Distortion Optimized 3D Gaussian Representation. In *European Conference on Computer Vision*, 2024.
- [49] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20310–20320, June 2024.
- [50] Yuanbo Xiangli, Linning Xu, Xingang Pan, Nanxuan Zhao, Anyi Rao, Christian Theobalt, Bo Dai, and Dahua Lin. Bungeenerf: Progressive neural radiance field for extreme multi-scale scene rendering. In *European conference on computer vision*, pages 106–122. Springer, 2022.
- [51] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction. *arXiv preprint arXiv:2309.13101*, 2023.
- [52] Zongxin Ye, Wenyu Li, Sidun Liu, Peng Qiao, and Yong Dou. AbsGS: Recovering fine details in 3d gaussian splatting. In *ACM Multimedia 2024*, 2024. URL <https://openreview.net/forum?id=tuch82hlmx>.
- [53] Taoran Yi, Jiemin Fang, Junjie Wang, Guanjun Wu, Lingxi Xie, Xiaopeng Zhang, Wenyu Liu, Qi Tian, and Xinggang Wang. GaussianDreamer: Fast Generation from Text to 3D Gaussians by Bridging 2D and 3D Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6796–6807, June 2024.
- [54] Alex Yu, Vickie Ye, Matthew Tancik, and Angjoo Kanazawa. pixelNeRF: Neural Radiance Fields from One or Few Images. In *CVPR*, 2021.

- [55] Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-Splatting: Alias-free 3D Gaussian Splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [56] Jiahui Zhang, Fangneng Zhan, Yingchen Yu, Kunhao Liu, Rongliang Wu, Xiaoqin Zhang, Ling Shao, and Shijian Lu. Pose-free neural radiance fields via implicit pose regularization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3534–3543, 2023.
- [57] Shijie Zhou, Zhiwen Fan, Dejia Xu, Haoran Chang, Pradyumna Chari, Tejas Bharadwaj, Suyu You, Zhangyang Wang, and Achuta Kadambi. DreamScene360: Unconstrained Text-to-3D Scene Generation with Panoramic Gaussian Splatting. In Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol, editors, *Computer Vision – ECCV 2024*, pages 324–342, Cham, 2025. Springer Nature Switzerland. ISBN 978-3-031-72658-3.

A Extended Derivation for Eq. 6

In Bayesian statistics, the posterior distribution of the parameters Θ given data $\mathcal{D}$ is defined as:

$$p(\Theta | \mathcal{D}) = \frac{p(\mathcal{D} | \Theta) p(\Theta)}{\underbrace{\int p(\mathcal{D} | \Theta) p(\Theta) d\Theta}_Z}, \quad (19)$$

where $p(\mathcal{D} | \Theta)$ is the likelihood, $p(\Theta)$ is the prior, and Z is the marginal likelihood.

In Eq. 5, we introduced a negative log-posterior

$$\mathcal{E}(\Theta) = \mathcal{L}(\Theta) + \lambda_v \sum_{v \in \mathcal{V}} \ln(1 + c_\Theta(v)) \quad (20)$$

In Eq. 3, we found out that $\ln p(\mathcal{D} | \Theta) \approx -\mathcal{L}(\Theta)$. Combining these two together, we can derive:

$$\mathcal{E}(\Theta) = -\ln p(\mathcal{D} | \Theta) - \ln p(\Theta) \quad (21)$$

If we exponentiate $-\mathcal{E}(\Theta)$:

$$e^{-\mathcal{E}(\Theta)} = p(\mathcal{D} | \Theta) p(\Theta), \quad (22)$$

we can derive the unnormalized posterior density. To turn this into a proper probability distribution, we normalize by $Z = \int e^{-\mathcal{E}(\Theta)} d\Theta$, yielding:

$$\pi(\Theta) = \frac{1}{Z} e^{-\mathcal{E}(\Theta)} = \frac{p(\mathcal{D} | \Theta) p(\Theta)}{Z} = p(\Theta | \mathcal{D}) \quad (23)$$

As $\pi(\Theta)$ matches the posterior distribution form in Eq. 19, we can define it as the posterior density.

B Extended Derivation for Eq. 17

Below is the derivation for Eq. 17. Recall Eq. 5, which defines the negative log-posterior, also known as the energy function:

$$\mathcal{E}(\Theta) = \mathcal{L}(\Theta) + \lambda_v \sum_{v \in \mathcal{V}} \ln(1 + c_\Theta(v)) \quad (24)$$

Thus $\Delta\mathcal{E} = \mathcal{E}(\Theta') - \mathcal{E}(\Theta)$ can be calculated as:

$$\begin{aligned} \Delta\mathcal{E} &= \mathcal{E}(\Theta') - \mathcal{E}(\Theta) \\ &= \mathcal{L}(\Theta') + \lambda_v \sum_{v \in \mathcal{V}} \ln(1 + c_{\Theta'}(v)) - (\mathcal{L}(\Theta) + \lambda_v \sum_{v \in \mathcal{V}} \ln(1 + c_\Theta(v))) \\ &= (\mathcal{L}(\Theta') - \mathcal{L}(\Theta)) + \lambda_v \sum_{v \in \mathcal{V}} [\ln(1 + c_{\Theta'}(v)) - \ln(1 + c_\Theta(v))] \end{aligned}$$

As we considered the case of having added exactly one Gaussian into voxel v' , we can define $c_{\Theta'}(v)$, the number of Gaussians in v' after the move, as:

$$c_{\Theta'}(v') = c_\Theta(v') + 1 \quad (25)$$

Moreover, the sum collapses to just the one term for voxel v' . Thus, continuing our derivation:

$$\begin{aligned} \Delta\mathcal{E} &= (\mathcal{L}(\Theta') - \mathcal{L}(\Theta)) + \lambda_v \sum_{v \in \mathcal{V}} [\ln(1 + c_\Theta(v') + 1) - \ln(1 + c_\Theta(v'))] \\ &= \Delta\mathcal{L} + \lambda_v \ln \left(\frac{1 + c_\Theta(v') + 1}{1 + c_\Theta(v')} \right) \\ &= \Delta\mathcal{L} - \ln \left(\frac{1 + c_\Theta(v')}{1 + c_\Theta(v') + 1} \right)^{\lambda_v} \end{aligned}$$

Setting $\left(\frac{1 + c_\Theta(v')}{1 + c_\Theta(v') + 1} \right)^{\lambda_v}$ as $D(v')$, we can derive:

$$\Delta\mathcal{E} = \Delta\mathcal{L} - \ln D(v') \quad (26)$$

Now set $x = \frac{1}{1+c_{\Theta}(v')}$. We can use the first-order Taylor series expansion (also known as the first-order Maclaurin expansion) to approximate

$$\lambda_v \ln(1+x) \approx \lambda_v x = \frac{\lambda_v}{1+c_{\Theta}(v')} \quad (27)$$

We can further approximate as follows:

$$\begin{aligned} \ln D(v') &= -\lambda_v \ln \left(1 + \frac{1}{1+c_{\Theta}(v')} \right) \\ &= -\lambda_v \ln(1+x) \approx -\lambda_v x \\ D(v') &\approx e^{-\lambda_v x} \end{aligned} \quad (28)$$

Using the Maclaurin expansion, we can approximate:

$$e^{-a} = \frac{1}{e^a} = \frac{1}{1+a+\frac{a^2}{2}+O(a^3)} \approx \frac{1}{1+a} \quad (29)$$

Thus we can approximate $e^{-\lambda_v x}$ as $\frac{1}{1+\lambda_v x}$

$$D(v') \approx e^{-\lambda_v x} \approx \frac{1}{1+\frac{\lambda_v}{1+c_{\Theta}(v')}} \approx \frac{1}{1+\lambda_v c_{\Theta}(v')} \quad (30)$$

C Extended Derivation for Surrogate

This section explains why we approximate $-\Delta\mathcal{L}$ to $I(i)$.

Let's say we insert a tiny splat $\{\varepsilon g_i\}$ to the current state Θ . We'll call this state Θ' . By the first-order term in the multivariate Taylor expansion in the direction of g_i :

$$\Delta\mathcal{L} = \mathcal{L}(\Theta') - \mathcal{L}(\Theta) \approx \left\langle \nabla_{g_i} \mathcal{L}(\Theta), \varepsilon g_i \right\rangle \quad (31)$$

where $\nabla_{g_i} \mathcal{L}(\Theta)$ is the gradient of the loss w.r.t the Gaussian's parameters and the inner product measures the first-order change when adding εg_i .

Since g_i only affects $p_i = [\Pi(\mathbf{x}_i)]$, we can show:

$$-\Delta\mathcal{L} \propto \alpha O(p_i) + \beta \text{SSIM}_{\text{agg}}(p_i) + \gamma \mathcal{L}_{\text{agg}}(p_i) = z_i \quad (32)$$

As stated in Eq. 10 and 11, we set:

$$I(i) = \sigma(z_i) = \frac{1}{1+e^{-z_i}} \quad (33)$$

The Maclaurin series for $\sigma(z) = \frac{1}{1+e^{-z}}$ around 0 is:

$$\begin{aligned} \sigma(z) &= \sigma(0) + \sigma'(0)z + \frac{\sigma''(0)}{2!}z^2 + \frac{\sigma^{(3)}(0)}{3!}z^3 + \dots \\ &= \frac{1}{2} + \frac{1}{4}z - \frac{1}{48}z^3 + \dots \approx \frac{1}{2} + \frac{1}{4}z \end{aligned}$$

Thus, $\sigma(z_i) \approx \frac{1}{2} + \frac{1}{4}z_i$. Since our Metropolis-Hastings rule uses only relative magnitudes of $\sigma(z_i)$, the constant and the scale are absorbed, yielding $\sigma(z_i) \propto z_i$. Finally, if we put everything together:

$$-\Delta\mathcal{L} \approx z_i \approx \sigma(z_i) = I(i) \quad (34)$$

D Comparison of the Number of Gaussians

Table 4 contrasts the number of Gaussians needed by our approach and by 3DGS for each scene. In every case, our method attains equal or superior rendering quality while requiring substantially fewer Gaussians.

Table 4: Comparison of our method and 3DGS on the number of Gaussian center points used to represent scenes in the Mip-NeRF 360 [1], Tanks & Temples [21], and Deep Blending [15] datasets.

Dataset	Mip-NeRF 360									Tanks&Temples		Deep Blending	
Method	Bonsai	Counter	Kitchen	Room	Bicycle	Flowers	Garden	Stump	Treehill	Train	Truck	Dr Johnson	Playroom
3DGS [18]	1,249,672	1,174,274	1,763,905	1,297,736	4,888,552	2,887,138	5,667,876	4,324,049	3,224,118	1,085,904	2,580,408	3,308,886	2,320,688
Ours	777,782	698,871	623,992	765,370	741,748	695,495	811,695	697,792	690,517	788,942	756,104	766,469	735,409

E Preliminary on Metropolis-Hastings Algorithm

The Metropolis-Hastings algorithm [33, 7] is a widely used Markov Chain Monte Carlo (MCMC) method designed to sample from complex, high-dimensional probability distributions. The objective of the algorithm is to construct a Markov chain whose stationary distribution $\pi(\mathbf{x})$ matches the target distribution, enabling efficient sampling where direct methods are impractical.

Given a target distribution $\pi(\mathbf{x})$ defined over a state space $\mathcal{X}$, the Metropolis-Hastings algorithm generates a sequence $\{\mathbf{x}^{(t)}\}_{t=1}^N$ of samples by iteratively proposing candidate points and accepting or rejecting them based on an acceptance probability. The transition from the current state $\mathbf{x}^{(t)}$ to a new candidate $\mathbf{y}$ is governed by a proposal distribution $q(\mathbf{x}, \mathbf{y})$.

At each iteration t , a new candidate $\mathbf{y}$ is proposed from the distribution $q(\mathbf{x}^{(t)}, \cdot)$. The acceptance probability is further computed as

$$\rho(\mathbf{x}^{(t)}, \mathbf{y}) = \min \left(1, \frac{\pi(\mathbf{y})q(\mathbf{y}, \mathbf{x}^{(t)})}{\pi(\mathbf{x}^{(t)})q(\mathbf{x}^{(t)}, \mathbf{y})} \right). \quad (35)$$

The candidate $\mathbf{y}$ is accepted with probability $\rho(\mathbf{x}^{(t)}, \mathbf{y})$; otherwise, the chain remains at the current state, $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)}$. The acceptance probability is designed to ensure that the Markov chain satisfies the detailed balance condition:

$$\pi(\mathbf{x})q(\mathbf{x}, \mathbf{y})\rho(\mathbf{x}, \mathbf{y}) = \pi(\mathbf{y})q(\mathbf{y}, \mathbf{x})\rho(\mathbf{y}, \mathbf{x}) \quad (36)$$

which guarantees that $\pi(\mathbf{x})$ is the stationary distribution of the Markov chain. The distribution of the generated samples converges to the target distribution $\pi(\mathbf{x})$ as the number of iterations N increases.

F Algorithms

Our algorithms of our coarse-fine Metropolis-Hastings sampling framework and overall pipeline are summarized in Algorithm 1 and 2.

G Per-Scene Results

In this section, we provide the full per-scene evaluations. We evaluated our method, 3DGS [18] and 3DGS-MCMC [20] on various datasets and scenes. We provide the per-scene results for Mip-NeRF 360 [1], Tank & Temples [21] and Deep Blending [15] datasets in Table 5 and 6. All evaluations have been conducted on a NVIDIA GeForce RTX 3090.

H Broader Impacts

As our method focuses on the core and theoretical problem of 3D Scene Reconstruction, our work has the potential for positive societal impact on various downstream applications using 3D Gaussian Splatting [18]. As we reduce the reliance on heuristics and the number of Gaussians used in a scene, we expect our work to enhance many downstream applications that require less memory usage. However, as we did not introduce any fundamental new content generation capabilities through our work, there is little potential for our work to have a negative societal impact beyond the ethical considerations already present in the field of 3D reconstruction.

Type of Dataset		Indoor				Outdoor				
Scene		Bonsai	Counter	Kitchen	Room	Bicycle	Flowers	Garden	Stump	Treehill
3DGS	PSNR	32.14	28.98	31.25	31.65	25.10	21.33	27.31	26.64	22.54
	SSIM	0.940	0.906	0.931	0.927	0.747	0.589	0.857	0.770	0.635
	LPIPS	0.206	0.202	0.117	0.197	0.244	0.359	0.122	0.216	0.347
	Train (mm:ss)	28:04	43:25	31:05	26:29	42:41	31:21	43:16	34:00	30:53
	# of Gaussians	1249672	1174274	1763905	1478813	5728256	3480107	5667876	4494105	3522801
	Storage (MB)	296.56	277.73	419.18	349.76	1354.80	823.09	1340.52	1062.91	833.18
3DGS-S	PSNR	31.37	28.84	30.91	31.04	23.55	20.10	26.27	24.84	22.64
	SSIM	0.935	0.909	0.924	0.918	0.616	0.488	0.794	0.677	0.590
	LPIPS	0.205	0.196	0.134	0.214	0.410	0.478	0.230	0.381	0.444
	Train (mm:ss)	26:42	28:04	33:59	25:22	25:54	23:58	28:17	23:02	28:00
	# of Gaussians	777782	698871	623992	764539	741748	695495	811695	697792	690517
	Storage (MB)	183.96	165.29	147.58	181.02	175.43	164.49	191.98	165.04	163.32
3DGS-MCMC	PSNR	32.48	29.27	31.41	32.15	25.13	21.64	26.71	27.21	23.09
	SSIM	0.949	0.920	0.930	0.933	0.745	0.595	0.832	0.791	0.648
	LPIPS	0.176	0.177	0.126	0.186	0.275	0.377	0.170	0.247	0.370
	Train (mm:ss)	32:08	37:27	30:25	35:37	31:56	25:39	24:16	23:50	26:01
	# of Gaussians	777782	698871	623992	764539	741748	695495	811695	697792	690517
	Storage (MB)	183.96	165.29	147.58	181.02	175.43	164.49	191.98	165.04	163.32
Ours	PSNR	32.52	29.30	31.35	32.15	24.53	20.99	26.49	25.95	22.80
	SSIM	0.950	0.918	0.930	0.934	0.711	0.556	0.819	0.743	0.621
	LPIPS	0.171	0.174	0.123	0.179	0.289	0.384	0.185	0.279	0.384
	Train (mm:ss)	41:05	46:59	41:20	46:31	34:58	35:13	31:36	32:46	37:05
	# of Gaussians	777782	698871	623992	764539	741748	695495	811695	697792	690517
	Storage (MB)	183.96	165.29	147.58	181.02	175.43	164.49	191.98	165.04	163.32

Table 5: Full per-scene results evaluated on Mip-NeRF 360 dataset.

Dataset		Tanks&Temples			Deep Blending		
Scene		Train	Truck	Average	Dr Johnson	Playroom	Average
3DGS	PSNR ($\uparrow$)	21.89	25.37	23.63	29.08	29.97	29.53
	SSIM ($\uparrow$)	0.814	0.882	0.848	0.901	0.907	0.904
	LPIPS ($\downarrow$)	0.207	0.147	0.177	0.244	0.244	0.244
	Train (mm:ss)	13:22	19:09	16:15	29:26	26:20	27:53
	# of Gaussians	1085904	2575516	1830710	3308886	2320688	2814787
	Storage (MB)	256.83	609.14	432.99	782.59	548.87	665.73
3DGS-S	PSNR ($\uparrow$)	21.80	24.95	23.38	28.99	29.54	29.27
	SSIM ($\uparrow$)	0.791	0.865	0.828	0.892	0.901	0.897
	LPIPS ($\downarrow$)	0.245	0.183	0.214	0.280	0.263	0.272
	Train (mm:ss)	19:16	18:05	18:41	20:18	20:23	20:21
	# of Gaussians	788942	756104	772523	766469	735409	750939
	Storage (MB)	186.60	178.83	182.72	181.28	173.93	177.61
3DGS-MCMC	PSNR ($\uparrow$)	22.58	26.02	24.30	29.37	30.30	29.84
	SSIM ($\uparrow$)	0.834	0.889	0.862	0.902	0.910	0.906
	LPIPS ($\downarrow$)	0.196	0.143	0.170	0.260	0.248	0.254
	Train (mm:ss)	15:50	15:45	15:48	26:28	23:39	25:03
	# of Gaussians	788942	756104	772523	766469	735409	750939
	Storage (MB)	186.60	178.83	182.72	181.28	173.93	177.61
Ours	PSNR ($\uparrow$)	22.44	25.54	23.99	29.77	30.47	30.12
	SSIM ($\uparrow$)	0.821	0.883	0.852	0.904	0.913	0.909
	LPIPS ($\downarrow$)	0.198	0.134	0.166	0.249	0.240	0.245
	Train (mm:ss)	20:38	19:40	20:09	32:27	29:54	31:10
	# of Gaussians	788942	756104	772523	766469	735409	750939
	Storage (MB)	186.60	178.83	182.72	181.28	173.93	177.61

Table 6: Full per-scene results evaluated on Tank&Temples and Deep Blending datasets.

Algorithm 1 Coarse–Fine Metropolis–Hastings Sampling for 3DGS

Require: Scene $\Theta = \{g_i\}$; error maps $(O, \text{SSIM}_{\text{agg}}, \mathcal{L}1_{\text{agg}})$; voxel size v ; density weight λ_v ; proposal stds (σ_c, σ_f) ; batch sizes (B_c, B_f) ; importance weights (α, β, γ) ; opacity threshold τ

```
1: for  $g_i \in \Theta$  where  $\alpha_i \leq \tau$  do
2:   RELOCATE( $g_i$ ) ▷ 1. Relocate Gaussians with low opacity
3: end for
4: for  $g_i \in \Theta$  do
5:    $p_i \leftarrow \Pi(\mathbf{x}_i)$ 
6:    $S(i) \leftarrow \alpha O(p_i) + \beta \text{SSIM}_{\text{agg}}(p_i) + \gamma \mathcal{L}1_{\text{agg}}(p_i)$ 
7:    $I(i) \leftarrow \sigma(S(i))$  ▷ 2. Per-Gaussian importance
8: end for
9:  $c(\cdot) \leftarrow \text{VOXELCOUNTS}(\Theta, v)$ 
10: for  $(\sigma, B) \in \{(\sigma_c, B_c), (\sigma_f, B_f)\}$  do ▷ 3. MH insertion: coarse  $\rightarrow$  fine
11:   Sample index set  $\mathcal{I}$  of size  $B$  with  $\Pr(i) \propto I(i)$ 
12:   for  $i \in \mathcal{I}$  do
13:      $\mathbf{x}' \leftarrow \mathbf{x}_i + \mathcal{N}(0, \sigma^2 \mathbf{I})$ 
14:      $v' \leftarrow \text{VOXELINDEX}(\mathbf{x}', v)$ 
15:      $D(v') \leftarrow 1/(1 + \lambda_v c(v'))$ 
16:      $\rho \leftarrow I(i) \cdot D(v')$  ▷  $\rho = \sigma(S(i)) D(v')$ 
17:     if  $\text{UNIFORM}(0, 1) < \rho$  then
18:        $\Theta \leftarrow \Theta \cup \{(\mathbf{x}', \Sigma_i, c_i, \alpha_i)\}$ 
19:     end if
20:   end for
21: end for
22: return  $\Theta$ 
```

Algorithm 2 Overall pipeline of Metropolis-Hastings 3DGS

```
1: Input:  $M, S, C, A$  (SfM Points, Covariances, Colors, Opacities); image dimensions  $w, h$ .
2: Output: Optimized and densified attributes  $M, S, C, A$ .
3:  $i \leftarrow 0$  ▷ Iteration count
4: while not converged do
5:    $\{V, \hat{I}\} \leftarrow \text{SAMPLETRAININGVIEW}()$  ▷ Camera  $V$  and image
6:    $I \leftarrow \text{RASTERIZE}(M, S, C, A, V)$ 
7:    $L \leftarrow \text{LOSS}(I, \hat{I})$ 
8:    $S_{\text{SSIM}, i} \leftarrow \text{SSIM}(I, \hat{I})$ 
9:    $S_{\text{L1}, i} \leftarrow \text{L1}(I, \hat{I})$ 
10:   $[M, S, C, A] \leftarrow \text{ADAM}(\nabla L)$  ▷ Update attributes
11:  if  $\text{ISREFINEMENTITERATION}(i)$  then
12:    for Current Scene  $\Theta$  do
13:      METROPOLIS_HASTINGS_SAMPLING( $M, S, C, A$ )
14:    end for
15:  end if
16:   $i \leftarrow i + 1$ 
17: end while
18: return  $M, S, C, A$ 
```

I Dataset Licenses

We used the following datasets:

- Mip-NeRF360 [1]: The license term is unavailable. Available at <https://jonbarron.info/mipnerf360/>
- Tanks and Temples [21]: Published under the Creative Commons Attribution 4.0 International (CC BY 4.0) License. Available at <https://www.tanksandtemples.org/license/>
- Deep Blending [15]: The license term is unavailable. Available at <http://visual.cs.ucl.ac.uk/pubs/deepblending/>